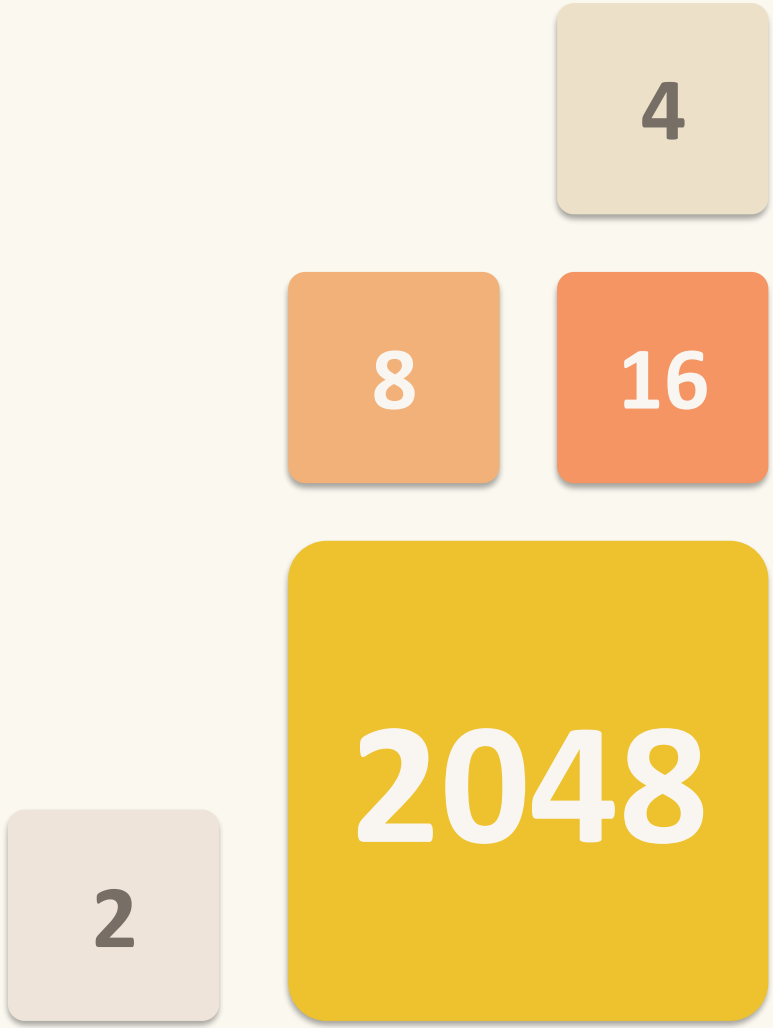
Florian Robrecht · Janin Jankovski · Anna Hartmann

Reinforcement learning with 2048

Machine Learning 2 · Assignment 5 · Group 5



A short history of 2048

Built in a weekend, viral in a week

2014
Year created

19
Age of creator

48 h
From idea to release

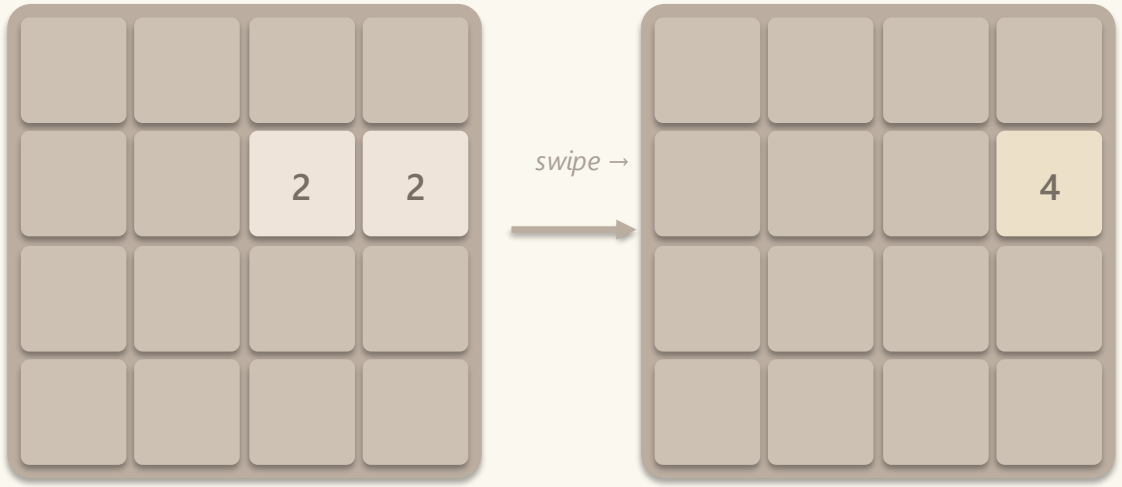
4 M+
Players in week 1

- Created by Gabriele Cirulli at age 19 (Milan) — open-source on GitHub, March 9, 2014
- Cirulli built it in about a day and a half as a personal challenge — he wanted to see if he could program a game from scratch
- Inspired by Threes! and the earlier 1024
- Theoretical maximum tile: 131,072 = 2^{17} — no one has reached it in fair play
- Within a week of launch, 2048 had been played by over 4 million people. Within a month, it hit ~100 million players
- Sparked an entire research literature on n-tuple TD learning (Szubert & Jaśkowski 2014 onwards)

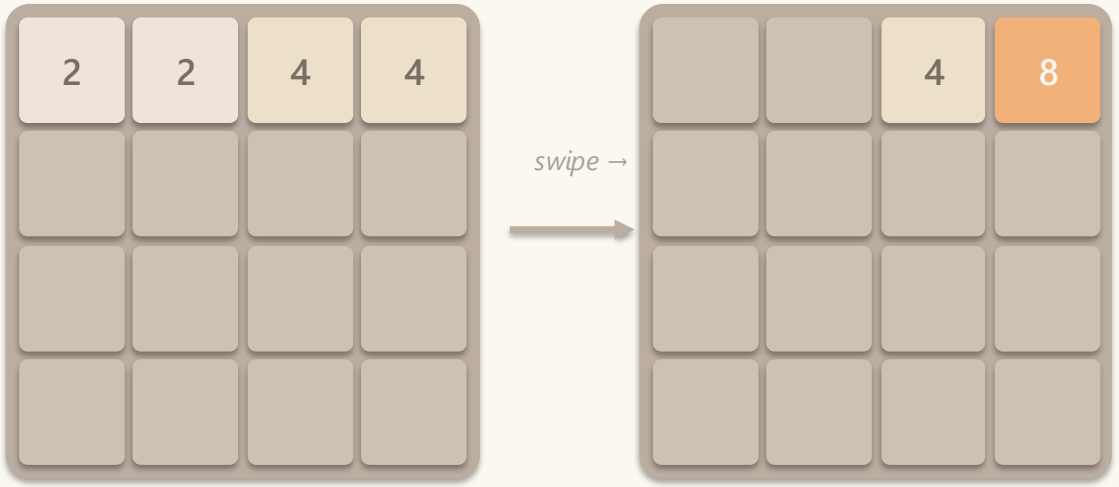
The rules

Slide, merge, repeat — until 2048 or until the board fills up

- 4×4 grid, swipe up, down, left or right · every tile slides as far as it can in that direction
- Two tiles with the same number that collide merge into one tile of double the value
- After every swipe a new 2 (90 %) or 4 (10 %) appears in a random empty cell
- Win at 2048 · Lose when the board is full with no legal merge



Simple merge: two 2s become a 4

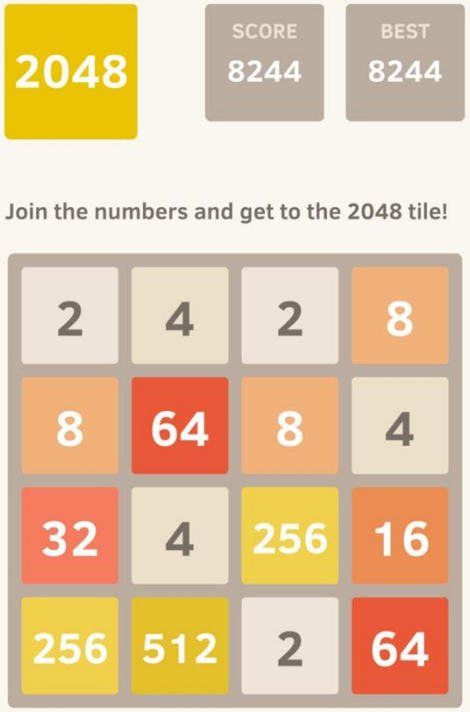


Two merges in one swipe (no chaining of new tiles)

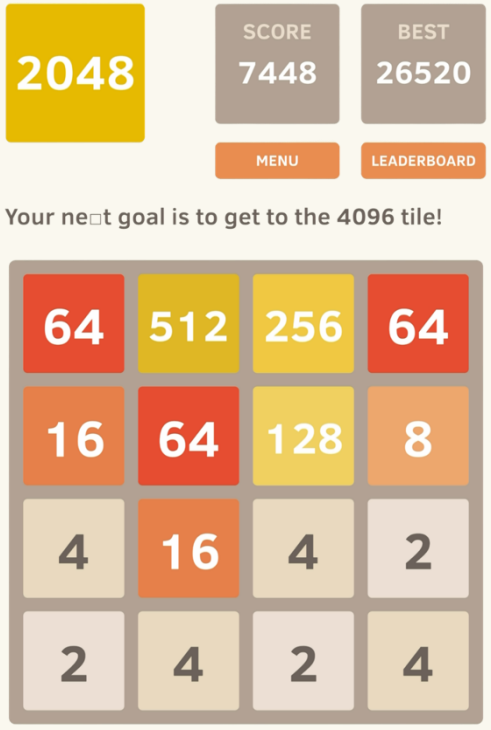
Our First Try

Anna, Flo & Janin

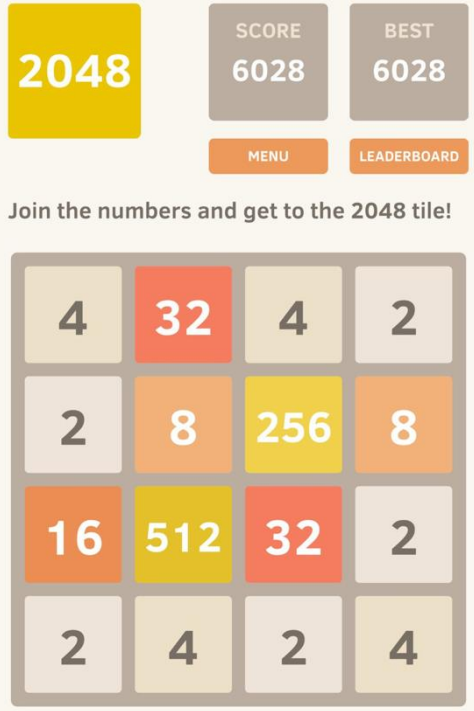
Anna



Flo

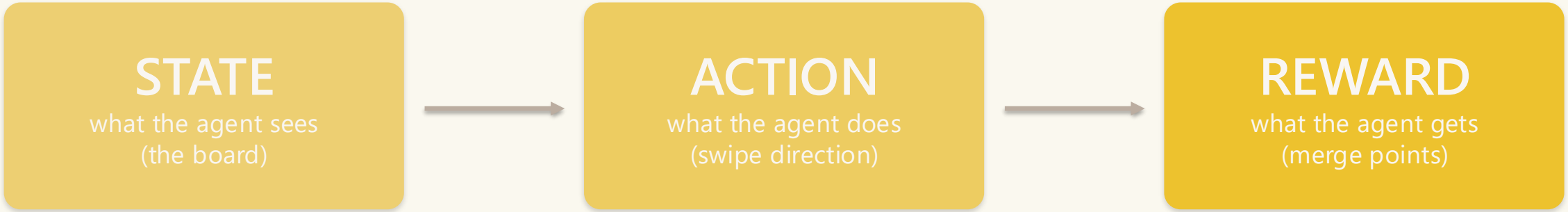


Janin



What is reinforcement learning?

An agent learns by trial and error — no labelled answers



The agent updates its strategy from rewards. No teacher, no labels.

- Every move gives the agent feedback (points scored on this move)
- Over many games the agent learns which moves tend to lead to high total scores
- The hard part: how to remember good moves when no two boards are ever quite the same

What we built

One game engine, four agents, one evaluation harness

Game engine
Tile stored as log base 2
65k-row Look Up Table
(unit-tested)

4 agents
Random · Greedy
DQN · N-tuple

Training
Separate loops
for DQN & N-tuple

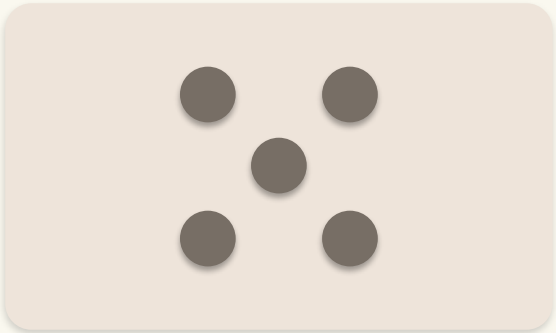
Evaluation
Hundreds of games
seeded test set

Every agent implements act(board, legal_mask) → swipe — so the evaluator can swap them freely

Same environment, same evaluation, same metrics → fair comparison

Meet the agents

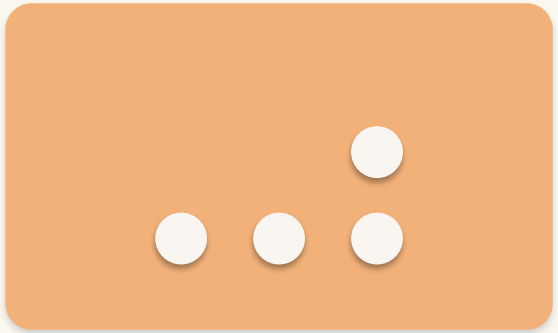
Two baselines · two learners



Random

Picks a legal swipe uniformly at random.

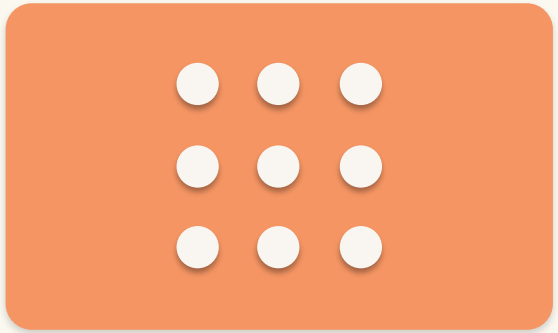
Arbitrary.



Greedy

Hand-coded heuristic: monotonicity + empty cells + corner bonus.

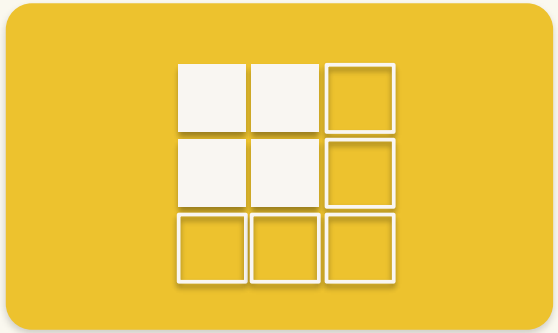
Human knowledge, no learning.



DQN

Deep Q-network. 256 → 512 → 256 → 4

Modern RL.



N-tuple

Lookup tables on 4 hand-picked board patches. Linear, symmetric.

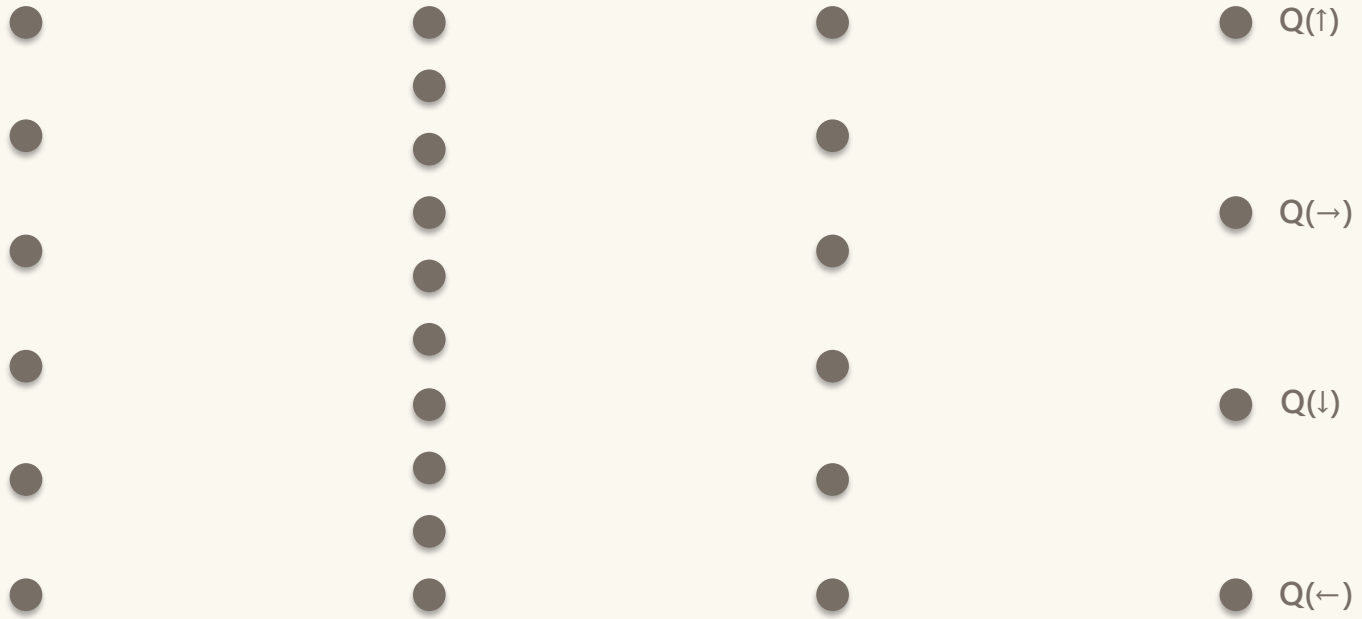
Classical 2048 method.

DQN — Deep Q-Network

A neural network predicts each swipe's long-term value

Board (4×4)

256
one-hot floats



256	512	256	4
input (one-hot)	*hidden ReLU*	*hidden ReLU*	*output Q-values*

The network sees 256 raw numbers. No domain knowledge — it has to discover everything itself.

Trained 5,000 games (~25 min on Apple-MPS)

DQN — a concrete forward pass

What the numbers actually look like for one board

Input board

One-hot encoding (256 floats)

```
Cell (0,0) = 8 (= 2³)
→ bucket 3 of 16:
[0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]

Cell (0,1) = 4 (= 2²)
→ bucket 2:
[0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]

… 16 cells × 16 buckets = 256 numbers fed into the network
```

Network outputs (Q-values)

Q(↑) = −1e9

Q(→) = +12.7

Q(↓) = +5.1

Q(←) = −1e9

→ Pick swipe RIGHT

In practice Q-values shift every gradient step as the network learns from its mistakes.

DQN — one training step

Play · store · sample · compute target · gradient step (×1,000,000)

Example
Q(s, →)

Example:
target =
3 + 0.99 · 20 = 22.8

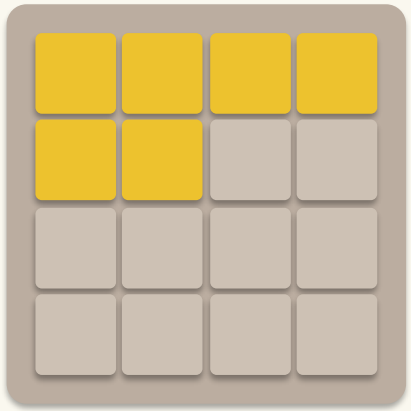
Reward now

Value of next board

Discount next board

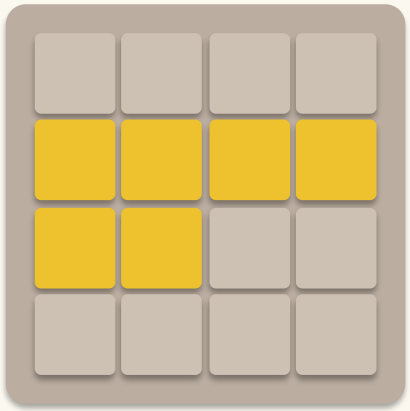
N-tuple TD — pattern-based value

Decompose the board into 4 small patches · sum 24 table reads



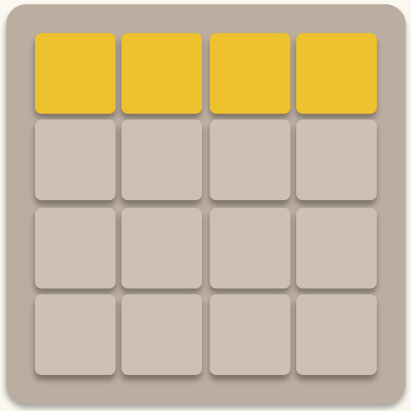
Pattern A

6-cell axe



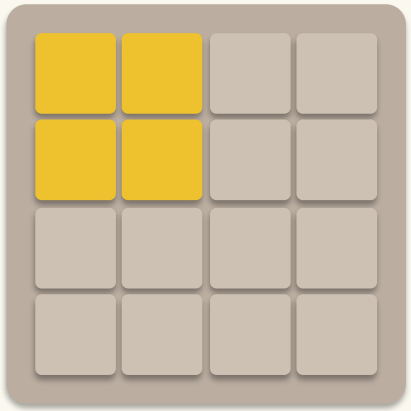
Pattern B

row-2 axe



Pattern C

top row (4 cells)



Pattern D

2×2 corner (4 cells)

V(board) = sum of all 24 lookup-table reads

- Each cell holds one of ~16 $\log_2$ values, so a 6-cell pattern has $16^6 \approx 16$ M-entry lookup table
- Weights are shared across all 8 board symmetries — one update teaches all rotated/reflected variants
- Total lookups per board: 8 + 8 + 4 + 4 = 24 table reads. No neural network, no gradient descent.

N-tuple — a concrete value computation

Read the patches · look up · sum

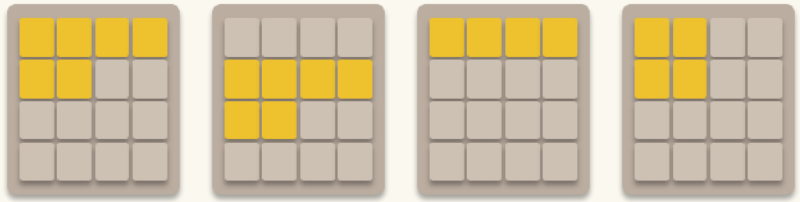
8	4	2	
4	2		
2			

Input board

Read $\log_2$ values at each patch

Pattern A	`[3, 2, 1, 0, 2, 1]`	*axe top-left*
Pattern B	`[2, 1, 0, 0, 1, 0]`	*axe row 2*
Pattern C	`[3, 2, 1, 0]`	*top row*
Pattern D	`[3, 2, 2, 1]`	*2×2 corner*

(× 8 symmetries for A and B, × 4 for C and D — 24 reads total)



Pattern A *6-cell axe* · **Pattern B** *row-2 axe* · **Pattern C** *top row (4 cells)* · **Pattern D** *2×2 corner (4 cells)*

Each read → one weight

w_A[idx_A] = +12.4

w_B[idx_B] = +3.1

w_C[idx_C] = +4.2

w_D[idx_D] = +6.5

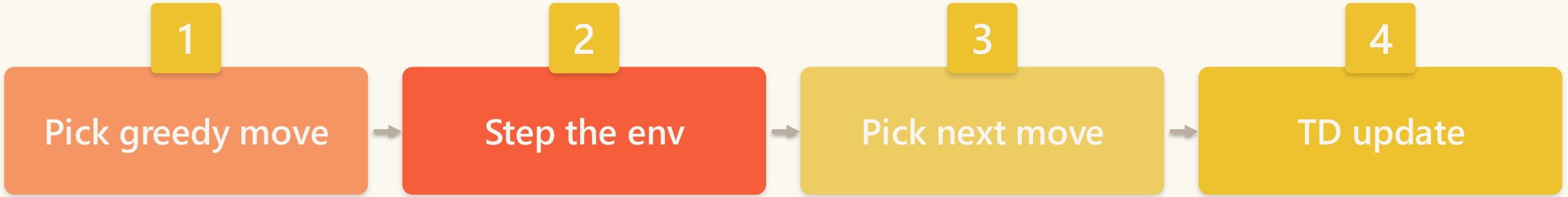
... (× 24 reads in total)

V(board) ≈ 47.5
(sum of all 24 lookups)

Numbers are illustrative. Real weights are learned by playing thousands of games. Based on (Szubert & Jaśkowski 2014 onwards)

N-tuple — one training step

Pick · step · pick · update — no buffer, no batches, no gradient descent



Try each legal swipe
Score by r + V(after)
Keep the best.

Get a_t, r_t, after_t.

Apply move → after_t
Spawn random tile
→ next board s_{t+1}

Repeat step 1 on s_{t+1}.
Get a_{t+1}, r_{t+1},
after_{t+1}.

δ = r_{t+1} + V(after_{t+1}) − V(after_t)

Add α·δ to all 24 weights that made up V(after_t).

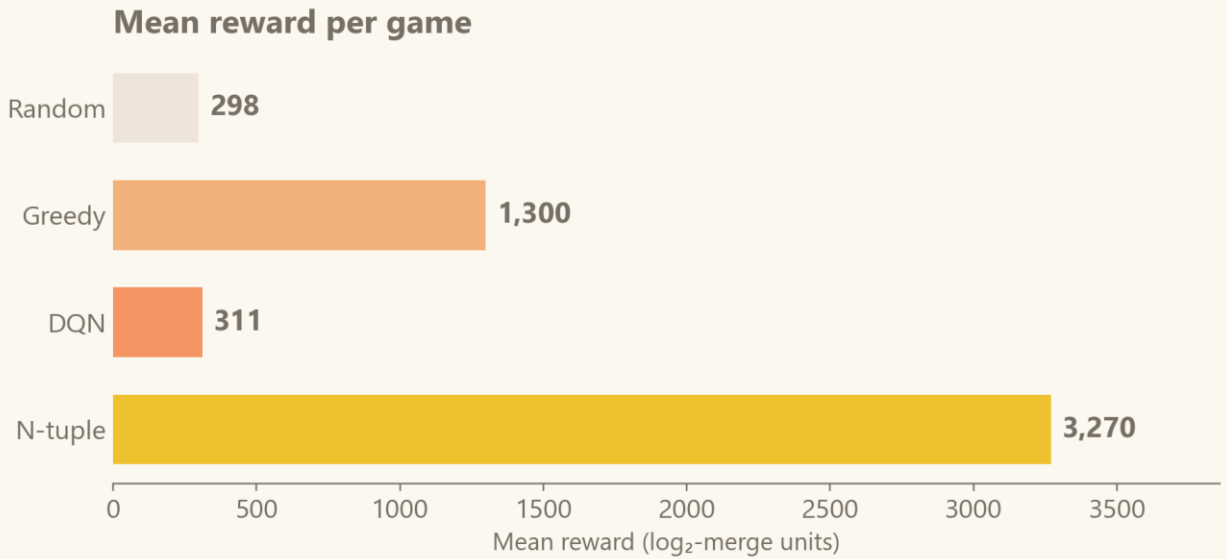
Each update touches 24 numbers — microseconds per step

Trained for 10,000 games (~55 min on CPU)

Results — the headline numbers

Random and greedy as baselines · DQN and N-tuple were trained

Random	Greedy	DQN	N-tuple
298	1,300	311	3,270
mean reward	mean reward	mean reward	mean reward



What stands out

- N-tuple wins by an order of magnitude on mean reward (3,270 vs 311 for DQN)
- DQN is effectively random: 311 vs 298 for uniform play
- Even greedy (no learning) beats DQN by 4×
- Scores are in $\log_2$-merge units — about 5× smaller than the official 2048 score

Why did the linear method win?

Three things the n-tuple gets for free — before it ever starts learning

Right features beat raw data

The n-tuple's patches encode exactly what good 2048 play is about:

- adjacency of tiles
- corner structure
- row monotonicity

DQN has 256 raw numbers and no idea any of that matters.

Symmetry sharing = free 4-8× efficiency

A clever move in the top-left is a clever move in the top-right (mirrored), bottom-left (flipped), etc.

The n-tuple updates all 8 variants at once.

DQN learns each corner separately.

Cleaner learning signal

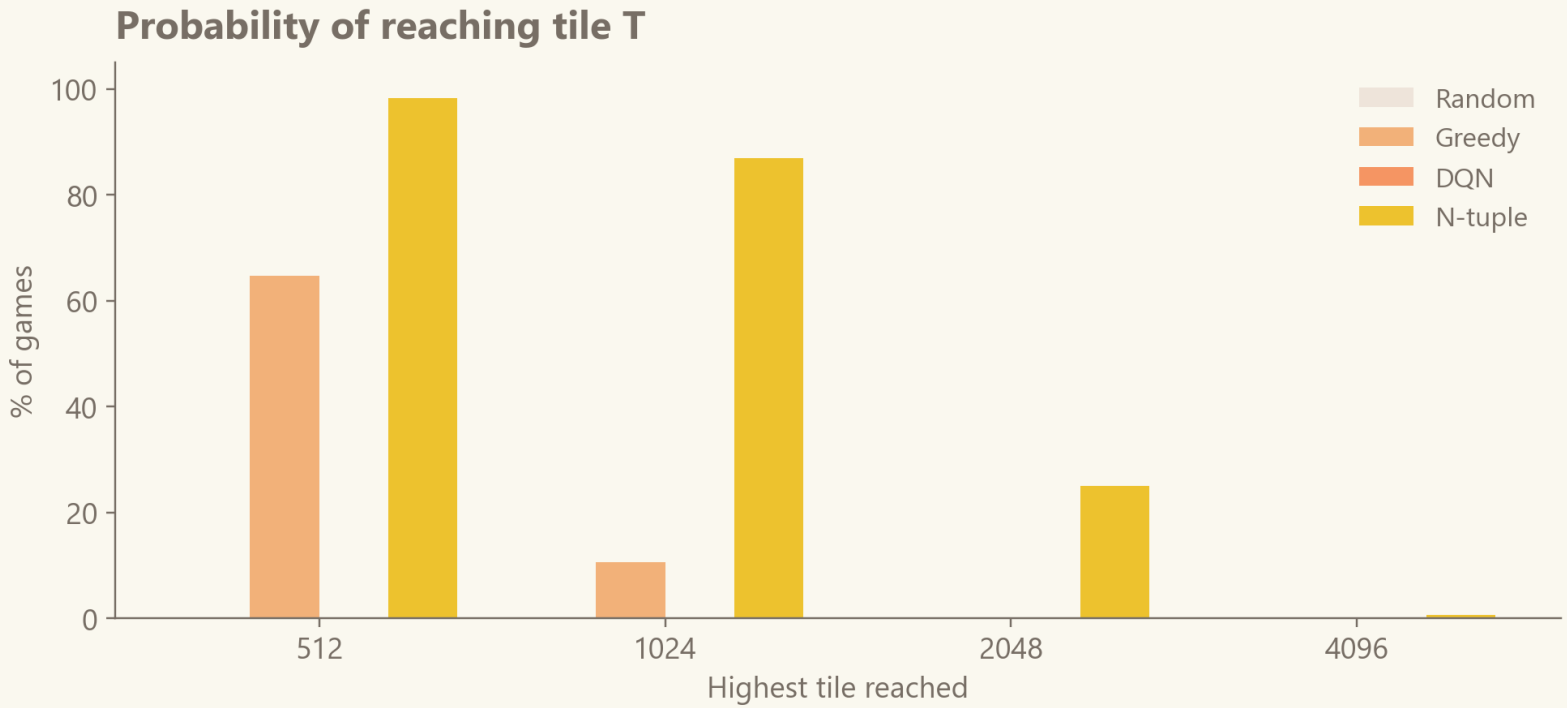
N-tuple learns afterstate values — the board before the random tile spawn.

That keeps spawn noise out of the target.

DQN learns the post-spawn board, so every target includes noise it couldn't have controlled.

Results — how often each tile is reached

Max-tile probability tells the real story



Reading the chart

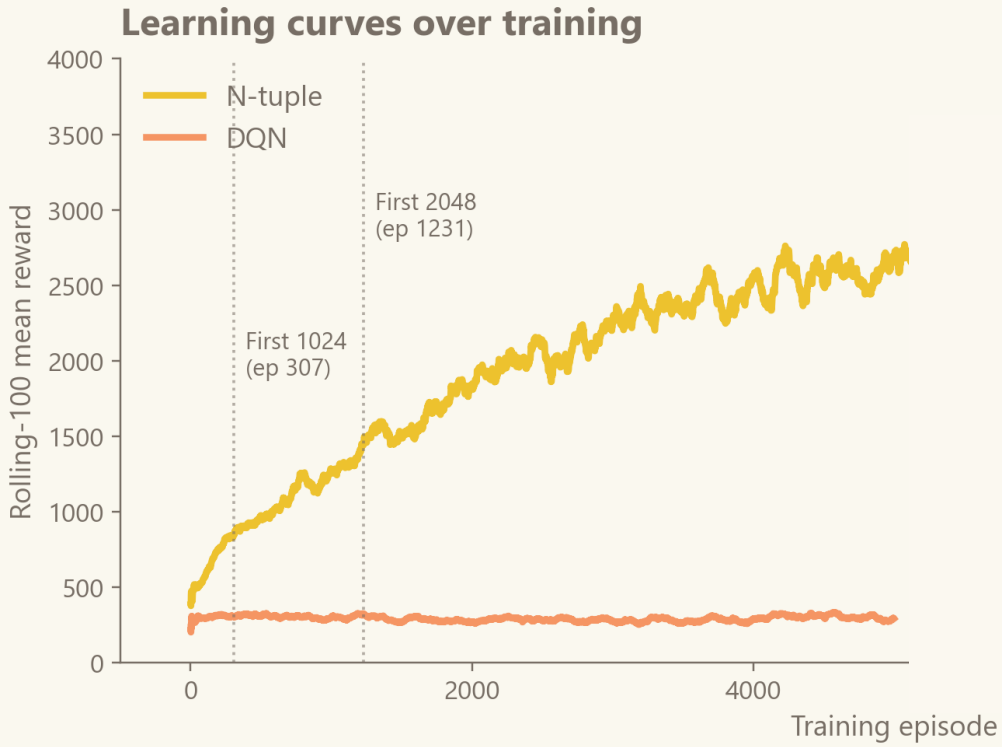
- N-tuple reaches 1024 in 87 % of games and 2048 in 25 %
- Greedy reaches 1024 in only 11 %, never 2048
- DQN never reaches 512 — same distribution as random
- The gap is not noise — it is structural

25 % of games end with the 2048 tile

(N-tuple, 300-game test set)

Results — how they learn over time

Rolling-100 mean reward across training episodes



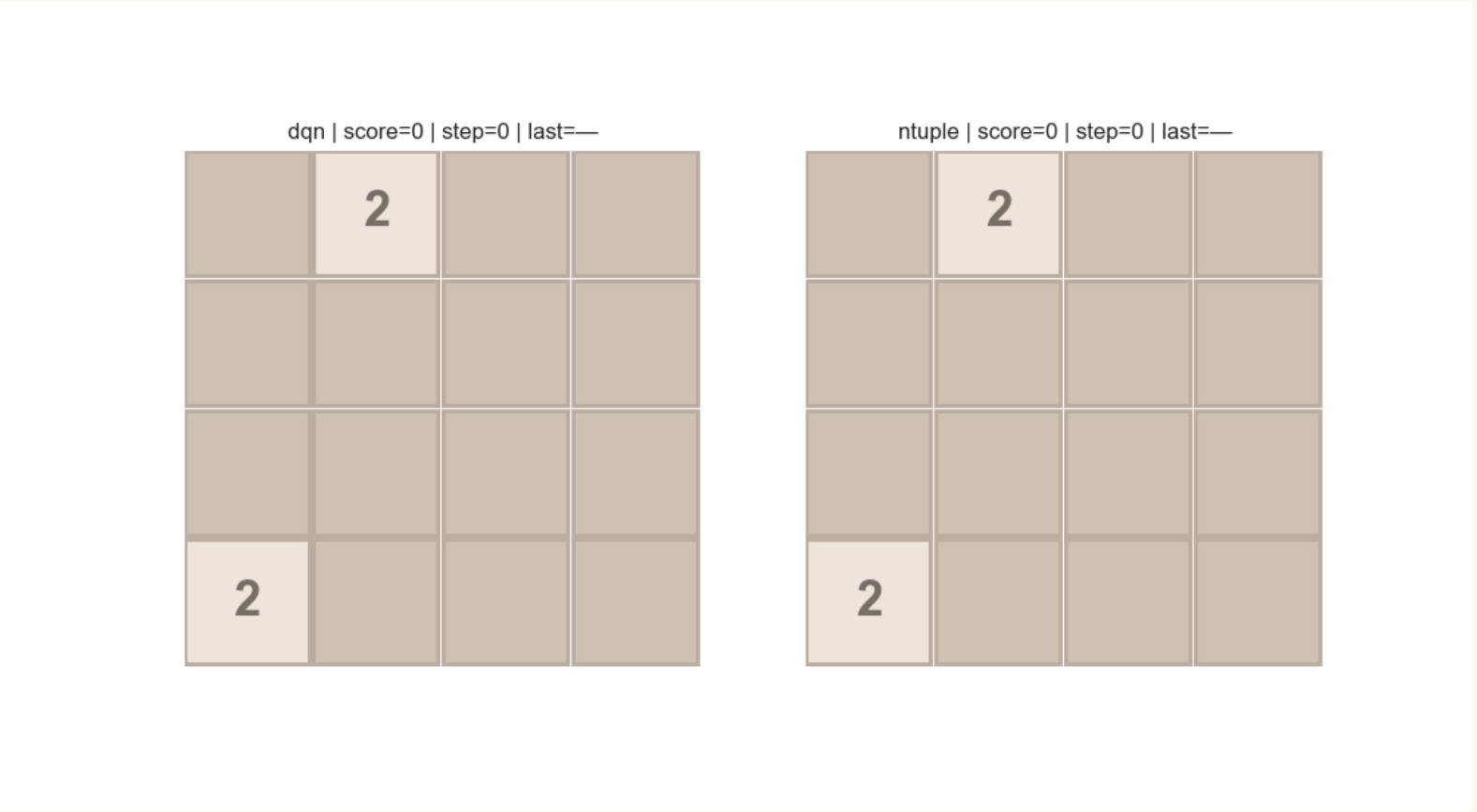
What to notice

- N-tuple climbs steadily from ~500 to ~3,000
- DQN rises from ~100 to ~300 and flatlines
- First 1024 tile: episode 307
- First 2048 tile: episode 1,231
- DQN does not show any learning

N-tuple's reward keeps growing · DQN's plateaus

Watch them play

DQN on the left · N-tuple on the right · same starting state



DQN dies quickly · N-tuple stacks tiles in a corner

Thanks for your attention

Questions?

2 4 16 8

Appendix — DQN in depth

Backup reference for questions: architecture, hyperparameters, and the five stabilisation tricks

Architecture & hyperparameters

- Network: fully-connected MLP — 256 → 512 → 256 → 4, ReLU
- Input: 16 board cells × 16 one-hot value buckets = 256 floats
- Output: 4 Q-values, one per swipe (↑ → ↓ ←)
- Loss: Smooth-L1 (Huber) — robust to outlier TD errors
- Optimiser: Adam, learning rate 5e-4 (= 0.0005)
- Discount factor: $\gamma = 0.99$
- Replay buffer: 100,000 transitions (ring buffer)
- Warm-up: 1,000 steps collected before learning starts
- Mini-batch: 512 transitions per gradient step
- Learn frequency: 1 gradient step every 4 environment steps
- Target-network sync: every 1,000 gradient steps
- Exploration: ε-greedy, ε decays 1.0 → 0.05
- Reward: $\log_2$ of the merged tile, then × 0.1 scaling
- Training budget: 5,000 games (≈ 25 min on Apple-MPS)

The five stabilisation tricks

1. **Target network** a frozen copy of the network computes the target, synced every 1,000 steps — stops the target from moving every step.
2. **Double-DQN** the online net picks the next action, the target net scores it — removes the upward bias of the max operator.
3. **Reward scaling** $\log_2$ encoding plus a × 0.1 multiplier — keeps rewards on a stable scale in both early and late game.
4. **ε-greedy exploration** act randomly with probability ε — forces the agent to keep exploring instead of only exploiting.
5. **Action masking** illegal swipes' Q-values are clamped to −1e9 — the agent never wastes a move on a no-op swipe.

Why the tricks are needed — basic DQN is unstable for three reasons: (1) it bootstraps off its own predictions, so the target keeps moving; (2) the max over noisy Q-estimates is biased upward; (3) reward magnitudes vary wildly across a game.